MammAlps Behaviour-Recognition Baseline

Week 1 Progress Report

Kostiantyn Boiar
MSc Dissertation, University of Glasgow

June 29, 2026

1 Objective and scope

The dissertation asks whether an animal's foreground *silhouette shape* improves multimodal behaviour recognition. Week 1 targets the mandatory first step: a trustworthy, reproducible **video-only** and **video+audio** baseline on the **MammAlps Benchmark I** dataset, scored with the authors' official evaluation script. The shape modality is added later; this week is about the foundation it bolts onto.

A hard design constraint shapes everything: the heavy encoders (VideoMAE for video, an AudioMAE-style model for audio) are **frozen**. We never train them; we run them once, cache their outputs to disk, and train only a small head on top. This is cheap and, crucially, keeps later comparisons fair (the silhouette stream is judged on information, not extra parameters).

2 Approach and strategy

The work is organised as a sequence of small steps, each with a concrete artifact and a check before moving on. Heavy, run-once feature extraction is designed for a **Kaggle free GPU** (run once); the light, repeatable head training and evaluation run on an **M4 laptop** (PyTorch MPS). Week 1 covers the setup work plus the first two checks (split verification and the evaluation-format check); no model has been trained yet.

3 Environment setup

- **Repository & version control.** Initialised git; created the `mammalps_b1/` source tree (config, data, encoders, models, scripts, utils) with a `.gitignore` that keeps data, caches, weights and virtual environments out of git. The project is on GitHub as `mambr`.
- **Local environment.** A dedicated `mammalps-env` virtual environment on Python 3.11 with PyTorch (MPS available), `transformers`, audio/video decoding and data libraries; `ffmpeg` installed. Dependencies pinned in `requirements-local.txt` and frozen to a lock file.
- **Official scorer vendored.** Copied `eval_b1.py` and `labels_mapping_b1.json` from the MammAlps repository at a pinned commit, so scoring is reproducible. A wrapper script (`run_eval.sh`) invokes it.
- **Kaggle templates.** A GPU setup-check cell and notes on staging the 88 GB dataset, ready for the heavy extraction step.
- **Reproducibility scaffolding.** A seeding utility and a data-free **smoke test** that builds the fusion head on random tensors and confirms the toolchain end to end. **done**

4 The evaluation contract and dataset facts

Before any modelling, the exact input/output contract of `eval_b1.py` was read from source and locked:

- Predictions are a tab-separated file, one row per (test clip × sampled segment), with all prediction columns before all label columns, each a bracketed float vector.
- Three tasks: **Species** (5 classes) and **Activity** (11) are multiclass; **Action** (19) is **multi-label**. This corrected an assumption in the original brief and determines the head's loss (sigmoid/BCE for actions, softmax/CE for the rest).
- The scorer groups rows by clip, averages the segments, and **asserts exactly 1244 test clips**. This assertion is our format check.

5 Data and splits

- **Data access without the 88 GB download.** The per-clip label/split CSVs live *inside* `mammalps_v1.zip`. Since the host supports HTTP range requests, a small fetcher pulls only those files (∼660 KB) instead of the whole archive.
- **Splits verified.** Parsed the comma-separated CSVs (`video_path, start_s, end_s, activity, actions, species`) into typed clip records with one-hot/multi-hot label vectors, and confirmed the counts: **train 4205 / val 686 / test 1244 / total 6135** — matching the paper exactly.
- **Findings.** The data is heavily imbalanced (the test set is ∼98% red deer; activities dominated by foraging and vigilance), and the contact/posture behaviours the project ultimately targets are rare (courtship 56, chasing 3, escaping 1). This justifies macro-mAP and class-balanced sampling, and is important context for the later per-behaviour analysis. **done**

6 Code architecture

The codebase is **object-oriented** and **fully config-driven**: every constant lives in a central `Config` (paths, task definitions, dataset facts, model dimensions, training hyper-parameters) with defaults overridable via YAML or environment variables. Feature code is class-based (`LabelSpace`, `SplitLoader`, `MultiModalHead`/`MultiTaskLoss`, and script classes such as `SplitChecker` and `MetadataFetcher`); only genuine utilities remain plain functions. Style is enforced with full type hints, Google-style docstrings, and `ruff` formatting/linting. Unrelated exploratory files from an earlier dataset (CalMS21) were removed so the repository is focused on MammAlps. **done**

7 Evaluation-format check

A `ResultsEmitter` class writes a *dummy* results file for all 1244 test clips (×10 segments): **real labels, random predictions**, in the exact column order and vector format the scorer expects. Running `eval_b1.py` on it confirmed the file passes the **1244-clip assertion** and produces metrics for all three tasks. The average mAP came out near-random (≈ **0.09**), exactly as expected for random guesses — proving the entire input/output pipeline is correct before any GPU time is spent. The emitter doubles as the skeleton for the inference script later. **done**

8 Frozen encoders: one-clip probe

The first step that touches the real video and audio. Two frozen encoder wrappers were written: a `VideoEncoder` (decodes 16 frames from a clip's mp4 and runs a frozen VideoMAE) and an `AudioEncoder` (loads the clip's wav and runs a frozen AST), each producing one mean-pooled embedding and never trained. A `ClipProbe` pulls a clip's mp4 and wav out of the 88 GB archive

with the same HTTP-range trick used for the CSVs (each ∼0.2 MB), then runs both encoders. On the M4 laptop (MPS) it produced finite **768-dimensional** video and audio embeddings for real clips. These dimensions ($D_v = D_a = 768$) are exactly what the feature cache and the fusion head will consume. A useful find: each clip ships with a *separate* audio wav, so no audio has to be demultiplexed from the video. One small fix was needed — the latest `torchaudio` routes `load` through a backend that was not installed, so wav loading was switched to `soundfile`. **done**

9 Reporting infrastructure

In parallel, the writing toolchain was set up: the Tectonic LaTeX compiler was installed and wired into the editor for live, build-on-save PDF previews. A compile-blocking bug in the proposal (an unclosed `\author`) was fixed, the affiliation corrected to the University of Glasgow throughout, and a full Glasgow-style dissertation skeleton drafted (title page, front matter, chapter structure, a written Background & Related Work chapter, and a bibliography). The report directory was then split into `proposal/`, `dissertation/` and `progress/` sections. **done**

10 Status and next steps

Item	Status
Environment, vendored scorer, smoke test	**done**
Splits parsed + verified (test=1244, total=6135)	**done**
Evaluation-format check (scorer accepts our output)	**done**
One-clip frozen-encoder probe (768-d video + audio)	**done**
Full feature cache on Kaggle (run once)	**next**
Train head: video, then video+audio (target: V+A > V)	—
Multi-seed report; lock the baseline	—

Immediately next. Run the same frozen encoders over all 6135 clips on a Kaggle GPU — sampling about ten windows per clip — and write the cached embeddings (one file per clip) plus a manifest, ready for the light head to train on.

Open risks / decisions.

- **Frozen vs fine-tuned.** The paper fine-tunes its encoder; our frozen-feature probe may score below its reported ∼0.45. The acceptance bar this step is a clean run plus *video+audio* > *video*; matching the exact number is a stretch goal.
- **Encoders.** Using nearest-available pretrained checkpoints rather than the paper's exact weights; absolute numbers may shift.
- **Kaggle storage.** The 88 GB dataset exceeds Kaggle's working disk; a staging strategy must be settled before the full extraction (shard-on-the-fly, or a hosted dataset).